



LAXMI CHARITABLE TRUST'S
SHETH L.U.J. COLLEGE OF ARTS & SIR M.V. COLLEGE OF
SCIENCE & COMMERCE
DR. S. RADHAKRISHNAN MARG, ANDHERI(EAST), MUMBAI - 400069

Name	Anshrah Shaikh
Roll No	T112
Class	T. Y. B. Sc. Computer Science
Semester	V
Subject	IKS In Computational Systems
Case Study On	Arthaśāstra-Inspired Polyalphabetic Symmetric Cipher System

Arthaśāstra-Inspired Polyalphabetic Symmetric Cipher System

An IKS-Based Approach to Symmetric Cryptography

(i) Problem Statement

Arthaśāstra Concept as a Symmetric Cryptographic Cipher System

The *Arthaśāstra*, an ancient Indian treatise on governance and statecraft, discusses methods of protecting sensitive information and communicating messages securely. This concept can be interpreted in the context of Computer Science as a **Polyalphabetic symmetric encryption system**, where the same secret key is used to transform plaintext into ciphertext and to recover the original message.

The objective of this project is to design and implement an **Arthaśāstra-inspired Polyalphabetic symmetric cipher** that applies a rule-based encryption and decryption mechanism to protect textual information. The system will demonstrate how an ancient Indian knowledge concept related to secrecy and secure communication can be modelled as a computational cryptographic algorithm.

(ii) Formal Specification of Rules/Algorithm

1. Cipher Description

The proposed **Arthaśāstra-Inspired Polyalphabetic Symmetric Cipher (AIPSC)** is a rule-based symmetric encryption system inspired by the principles of secrecy and protection of sensitive communication discussed in the *Arthaśāstra*.

The cipher uses a **shared secret key** for both encryption and decryption. It combines a repeating key-based substitution mechanism with a positional shift to transform plaintext into ciphertext.

The system operates on the English alphabet using modulo-26 arithmetic.

2. Alphabet Representation

Each alphabetic character is assigned a numerical value as follows:

$$A = 0, \quad B = 1, \quad C = 2, \quad \dots, \quad Z = 25$$

Let the plaintext be represented as:

$$P = P_1 P_2 P_3 \dots P_n$$

where P_i represents the i th alphabetic character of the plaintext.

Similarly, the ciphertext is represented as:

$$C = C_1 C_2 C_3 \dots C_n$$

where C_i represents the corresponding ciphertext character.

3. Secret Key

Let the secret key be:

$$K = K_1 K_2 \dots K_m$$

where m is the length of the key.

If the plaintext contains more alphabetic characters than the length of the key, the key is repeated cyclically.

The key character used for position i is therefore:

$$K_i = K_{((i-1) \bmod m) + 1}$$

This ensures that every alphabetic character in the plaintext has a corresponding key character.

4. Positional Shift

In addition to the key-based shift, the algorithm introduces a positional shift.

For the i th alphabetic character:

$$S_i = i$$

Thus, the first alphabetic character has a positional shift of 1, the second has a shift of 2, and so on.

This produces a progressively changing shift throughout the message.

5. Encryption Rule

Let:

- P_i = numerical value of the plaintext character
- K_i = numerical value of the corresponding key character
- S_i = positional shift
- C_i = numerical value of the resulting ciphertext character

The encryption operation is defined as:

$$C_i = (P_i + K_i + S_i) \bmod 26$$

The resulting numerical value C_i is then converted back into its corresponding alphabetic character.

The operation is repeated for every alphabetic character in the plaintext.

6. Decryption Rule

Since the system is symmetric, the **same secret key** used during encryption is required for decryption.

The plaintext value is recovered using:

$$P_i = (C_i - K_i - S_i) \bmod 26$$

The resulting value is converted back into its corresponding alphabetic character.

Therefore, the encryption and decryption operations satisfy:

$$D_K(E_K(P)) = P$$

where:

- EK represents encryption using key K
 - DK represents decryption using key K
 - P represents the original plaintext.
-

7. Character Handling Rules

The following rules are applied while processing the message:

1. Alphabetic characters are encrypted using the defined mathematical rules.
 2. Uppercase letters remain uppercase.
 3. Lowercase letters remain lowercase.
 4. Spaces are preserved without modification.
 5. Numbers are preserved without modification.
 6. Special characters and punctuation marks are preserved without modification.
 7. The positional count i is incremented **only for alphabetic characters**.
 8. The secret key is applied cyclically to the alphabetic characters.
-

8. Encryption Algorithm

Algorithm 1: Arthaśāstra-Inspired Symmetric Encryption

Input: Plaintext P, Secret Key K

Output: Ciphertext C

1. Read the plaintext P and secret key K.
2. Validate that the key contains at least one alphabetic character.
3. Convert the alphabetic characters of the key into numerical values using $A=0, \dots, Z=25$.
4. Initialize the alphabetic character position $i=1$.
5. Traverse the plaintext from left to right.
6. If the current character is alphabetic:
 - Convert it to its numerical value P_i .
 - Select the corresponding key value K_i cyclically.
 - Set the positional shift $S_i=i$.
 - Calculate:

$$C_i = (P_i + K_i + S_i) \bmod 26$$

- Convert C_i back to a letter.
 - Preserve the original letter case.
 - Increment i .
7. If the current character is not alphabetic, copy it directly to the ciphertext.
 8. Return the generated ciphertext.
-

9. Decryption Algorithm

Algorithm 2: Arthaśāstra-Inspired Symmetric Decryption

Input: Ciphertext C, Secret Key K

Output: Plaintext P

1. Read the ciphertext C and the same secret key K.
2. Convert the alphabetic characters of the key into numerical values.
3. Initialize the alphabetic character position $i=1$.
4. Traverse the ciphertext from left to right.
5. If the current character is alphabetic:
 - Convert it to its numerical value C_i .
 - Select the corresponding key value K_i cyclically.
 - Set the positional shift $S_i=i$.
 - Calculate:

$$P_i = (C_i - K_i - S_i) \bmod 26$$

- Convert P_i back to a letter.
 - Preserve the original letter case.
 - Increment i .
6. If the current character is not alphabetic, copy it directly to the plaintext.
 7. Return the recovered plaintext.

10. Example of the Rule

Consider the plaintext:

$$P = \text{HELLO}$$

and the secret key:

$$K = \text{RAM}$$

The key is repeated cyclically:

$$\text{R A M R A}$$

Using:

$$A = 0, \quad B = 1, \quad \dots, \quad Z = 25$$

we obtain:

$$H = 7, \quad E = 4, \quad L = 11, \quad L = 11, \quad O = 14$$

and:

$$R = 17, \quad A = 0, \quad M = 12$$

The positional shifts are:

$$S_1 = 1, \quad S_2 = 2, \quad S_3 = 3, \quad S_4 = 4, \quad S_5 = 5$$

For the first character:

$$C_1 = (7 + 17 + 1) \bmod 26$$

$$C_1 = 25$$

which corresponds to:

$$C_1 = Z$$

The same procedure is applied to the remaining characters to produce the complete ciphertext.

11. Symmetric Property

The system is classified as a **symmetric cipher** because encryption and decryption require the same shared secret key:

$$K_{\text{encryption}} = K_{\text{decryption}}$$

The correctness of the algorithm can be expressed as:

$$D_K(E_K(P)) = P$$

Thus, when the correct key is supplied, decrypting an encrypted message should always recover the original plaintext.

(iii) Working Code

```
# Arthaśāstra-Inspired Polyalphabetic Symmetric Cipher
# AIPSC

def prepare_key(key):
    """Remove non-alphabetic characters and convert key to uppercase."""
    return ''.join(char.upper() for char in key if char.isalpha())

def encrypt(plaintext, key):
    """Encrypt plaintext using the AIPSC algorithm."""

    key = prepare_key(key)

    if not key:
        raise ValueError("Key must contain at least one alphabetic character.")

    ciphertext = ""
    key_index = 0
    position = 1

    for char in plaintext:

        if char.isalpha():
            p = ord(char.upper()) - ord('A')

            k = ord(key[key_index % len(key)]) - ord('A')

            s = position

            #  $C_i = (P_i + K_i + S_i) \bmod 26$ 
            c = (p + k + s) % 26

            encrypted_char = chr(c + ord('A'))

            if char.islower():
                encrypted_char = encrypted_char.lower()

            ciphertext += encrypted_char

            key_index += 1
            position += 1

        else:
            ciphertext += char

    return ciphertext
```

```

def decrypt(ciphertext, key):
    """Decrypt ciphertext using the AIPSC algorithm."""

    key = prepare_key(key)

    if not key:
        raise ValueError("Key must contain at least one alphabetic character.")

    plaintext = ""
    key_index = 0
    position = 1

    for char in ciphertext:

        if char.isalpha():
            c = ord(char.upper()) - ord('A')

            k = ord(key[key_index % len(key)]) - ord('A')

            s = position

            #  $P_i = (C_i - K_i - S_i) \bmod 26$ 
            p = (c - k - s) % 26

            decrypted_char = chr(p + ord('A'))

            if char.islower():
                decrypted_char = decrypted_char.lower()

            plaintext += decrypted_char

            key_index += 1
            position += 1

        else:
            plaintext += char

    return plaintext

# -----
# Main Program
# -----

print("=" * 63)
print("  ARTHAŚĀSTRA-INSPIRED POLYALPHABETIC SYMMETRIC CIPHER (AIPSC)")
print("=" * 63)

while True:

```

```

print("\n1. Encrypt")
print("2. Decrypt")
print("3. Exit")

choice = input("\nEnter your choice: ")

if choice == "1":

    plaintext = input("Enter plaintext: ")
    key = input("Enter secret key: ")

    try:
        ciphertext = encrypt(plaintext, key)

        print("\nCiphertext:", ciphertext)

    except ValueError as e:
        print("Error:", e)

elif choice == "2":

    ciphertext = input("Enter ciphertext: ")
    key = input("Enter secret key: ")

    try:
        plaintext = decrypt(ciphertext, key)

        print("\nDecrypted plaintext:", plaintext)

    except ValueError as e:
        print("Error:", e)

elif choice == "3":

    print("\nExiting program...")
    break

else:
    print("\nInvalid choice. Please enter 1, 2 or 3.")

```

Working Principle – Example

The working of the proposed cipher can be understood through a simple example. Consider the plaintext “Anshrah” and the secret key “Enor”.

Since the message contains more characters than the key, the key is repeated cyclically:

Plaintext: A n s h r a h

Key: E n o r E n o

Position: 1 2 3 4 5 6 7

The cipher processes each alphabetic character using its corresponding key character and its position in the message. The key provides one shift, while the position of the character provides an additional shift.

For example, the first character **A** is paired with **E** and is at position **1**. These are combined to produce **F**. The same process is then applied to every remaining character.

The transformation can be summarized as:

Plaintext	Plaintext Position (P_i)	Key	Key Position (K_i)	Position (S_i)	$C_i = (P_i + K_i + S_i) \bmod 26$	(C_i)	Ciphertext
A	0	E	4	1	$(0+4+1) \bmod 26$	5	F
n	13	N	13	2	$(13+13+2) \bmod 26$	2	c
s	18	O	14	3	$(18+14+3) \bmod 26$	9	j
h	7	R	17	4	$(7+17+4) \bmod 26$	2	c
r	17	E	4	5	$(17+4+5) \bmod 26$	0	a
a	0	N	13	6	$(0+13+6) \bmod 26$	19	t
h	7	O	14	7	$(7+14+7) \bmod 26$	2	c

Thus, the plaintext: **Anshrah**, is transformed into the ciphertext: **Fcjcatc**

Anshrah → Fcjcatc

For decryption, the same secret key “**Enor**” is provided to the system. The encryption steps are reversed, allowing the original plaintext “**Anshrah**” to be recovered.

This demonstrates the symmetric nature of the proposed cipher, as the **same secret key is required for both encryption and decryption**.

Output shows how the Plaintext was encrypted and Decrypted:

```
=====
ARTHASĀSTRA-INSPIRED POLYALPHABETIC SYMMETRIC CIPHER (AIPSC)
=====

1. Encrypt
2. Decrypt
3. Exit

Enter your choice: 1
Enter plaintext: Anshrah
Enter secret key: Enor

Ciphertext: Fcjcatc

1. Encrypt
2. Decrypt
3. Exit

Enter your choice: 2
Enter ciphertext: Fcjcatc
Enter secret key: Enor

Decrypted plaintext: Anshrah
```

(iv) Test Cases

The proposed Arthaśāstra-Inspired Polyalphabetic Symmetric Cipher (AIPSC) was tested using different plaintexts and secret keys to verify its encryption and decryption functionality. The test cases include normal text, different key lengths, spaces, numbers, punctuation, mixed-case input, and edge cases.

Test Case	Plaintext	Secret Key	Expected Ciphertext	Decrypted Output	Result
1	Anshrah	Enor	Fcjcac	Anshrah	Pass
2	HELLO	KEY	RGCYS	HELLO	Pass
3	hello	key	rgcys	hello	Pass
4	Hello World	ABC	Jgopt Xnvmr	Hello World	Pass
5	Computer Science	CS	Gmktjvkx Uqjfmh	Computer Science	Pass
6	Secret Message!	CODE	Vqgukq Rqjfxw!	Secret Message!	Pass
7	Hello123	KEY	Rgcys123	Hello123	Pass
8	Hello, World!	RAM	Zfqtw, Zmbvh!	Hello, World!	Pass
9	A	B	C	A	Pass
10	Test@123	ABC	Uftx@123	Test@123	Pass
11	Chase the Shadows and call it light	Lux	Odahd wzg Ycfmmee bys gods zo vceug	Chase the Shadows and call it light	Pass
12	Encryption	Decrypt	Ithmbkttbz	Encryption	Pass

Test Case Observations

The test cases demonstrate that:

- The same secret key successfully encrypts and decrypts the message.
- Keys shorter than the plaintext are repeated cyclically.
- Uppercase and lowercase characters retain their original case.
- Spaces and punctuation marks remain unchanged.
- Numbers are preserved without encryption.
- A single-character plaintext can also be processed.
- Different key lengths can be used successfully.
- The decrypted output matches the original plaintext when the correct key is supplied.

All test cases successfully returned the original plaintext after decryption, indicating that the implemented encryption and decryption rules are functioning as specified.

(v) Analysis of Correctness and Limitations

Correctness

The proposed AIPSC algorithm correctly performs both encryption and decryption using the same secret key. During encryption, each alphabetic character is transformed using the corresponding repeating key character and its position in the message. During decryption, these operations are reversed using the same key.

The test cases demonstrate that the decrypted output matches the original plaintext for different types of input, including uppercase and lowercase text, spaces, numbers, punctuation, and different key lengths. Therefore, the implementation follows the specified rules and satisfies the basic requirement of a symmetric cipher system.

The correctness of the system can be expressed as:

$$D_K(E_K(P)) = P$$

where (P) is the original plaintext and (K) is the shared secret key.

And:

$$P \xrightarrow{E_K} C \xrightarrow{D_K} P$$

where P represents the original plaintext, C represents the ciphertext, and K represents the shared secret key.

Limitations

Although the proposed cipher demonstrates the concept of symmetric encryption, it is **not suitable for protecting real-world sensitive information**. It is a simple educational cipher and does not provide the level of security offered by modern cryptographic algorithms.

Its main limitations are:

1. **Limited security:** The cipher uses simple mathematical transformations and can potentially be broken through cryptanalysis.
2. **Repeating key:** A short key is repeated throughout the message, which can reveal patterns in longer ciphertexts.
3. **Predictable positional shift:** The additional shift follows a predictable sequence based on the character position.
4. **No authentication:** The system only encrypts data and does not verify whether the ciphertext has been modified.
5. **Limited character support:** The current implementation is designed primarily for alphabetic English characters, while other characters are preserved rather than encrypted.
6. **Not intended for real-world cryptography:** The algorithm is primarily designed to demonstrate the mapping of an IKS concept to a Computer Science concept.

Therefore, the proposed cipher is best considered an **educational and conceptual implementation**, rather than a replacement for established modern encryption standards.